plan.md - FLIPSIDE Development Plan

Project: FLIPSIDE **Developer:** Gift Zowa **Student Number:** ST10443108 **Engine:** Unity 2022.3 LTS (URP) **Jam Duration:** 3 Days (March 2026)

AI Tools Used

Purpose	Tool
Code generation & refactoring	Cursor (AI-assisted IDE)
2D art & tile sprites	Leonardo AI
Music & ambience	Soundraw / AIVA
Sound effects	ElevenLabs / GizAI
Writing & documentation	Claude (Anthropic)

Milestones

Day 1 - Ideation & Planning

- [x] Interpret theme ("Switch") and select concept
- [x] Refine concept: relay mechanic - one character at a time, Tab to switch, physical Switch objects
- [x] Write High Concept Document
- [x] Create plan.md, readme.md, requirements.txt, refinements-changes.md
- [x] Write all core scripts with Cursor assistance
- [x] Set up Unity 2022.3 LTS project (URP 2D template)
- [x] Install required packages via Package Manager
- [x] Enable New Input System
- [x] Create tags (ARC, ECHO) and layer (Ground)
- [x] Set up Git repository and initial commit
- [x] Create Input Actions asset (FlipsideInputActions)
- [x] Create .cursorrules file for Cursor context
- [x] Build and test core Tab switching mechanic
- [x] Generate first tile art sets using Leonardo AI

Day 2 - Prototype Build

- [x] Set up split-screen cameras (SplitScreenSetup + CameraFollow on each camera)
- [x] Build ARC and ECHO GameObjects with correct components and tags
- [x] Confirm Tab switching works with camera dimming
- [x] Build Levels 1-4 in Tilemap editor
- [x] Implement WorldSwitch and SwitchableObject system
- [x] Implement ExitDoor logic (auto-switches to other character on activation)
- [x] Set up Canvas and UIManager (level label, flash overlay, dividing line)
- [x] Set up LevelCompleteUI panel
- [x] Add dual timer system (LevelTimer) with active/inactive speed differential
- [x] Integrate AI-generated background music
- [x] Record Day 2 gameplay video (MP4, narrated)

Day 3 - Refinement & Submission

- [x] Build Levels 5-7
- [x] Add Boost Orb system (BoostOrb.cs + ApplyBoost on PlayerController)
- [x] Add Moving Hazards (MovingHazard.cs with ghost trail)
- [x] Polish UIManager: camera shake on death, label punch, triple-flicker dividing line
- [x] Polish LevelTimer: frost icons, vertex wave on inactive timer, strobe on critical
- [x] Polish LevelCompleteUI: bounce slide-in, elastic header pop, button fade
- [x] Polish ExitDoor: scale punch + continuous pulse after activation
- [x] Polish HazardTile: flash effect on contact
- [x] Polish CameraFollow: predictive look-ahead
- [x] Build MainMenu scene (title float, subtitle fade, button slide-in)
- [x] Fix UI responsive anchoring (all elements anchored correctly for any resolution)
- [x] Set Build Settings scene order
- [x] Final Windows build
- [x] Record Day 3 showcase video (MP4, narrated)
- [x] Fill in all reflection prompts
- [x] Finalise all documentation
- [x] Zip project folder and submit to ARC with Git link

Complete Script List

Script	Status	Description
PlayerController.cs	Done	Move, jump, coyote time, jump buffer, active/inactive freeze, boost system, outline effect
EchoController.cs	Done	Inherits PlayerController, identical controls
CharacterSwitcher.cs	Done	Tab switching, asymmetric camera brightness transition
WorldSwitch.cs	Done	Physical switch - activates SwitchableObjects in other world
SwitchableObject.cs	Done	Toggle/TurnOn/TurnOff - platforms, hazards, gates
SplitScreenSetup.cs	Done	Two cameras, viewport rects top/bottom
CameraFollow.cs	Done	Smooth follow, predictive look-ahead, optional Y tracking
HazardTile.cs	Done	Flash on contact, triggers level reset
MovingHazard.cs	Done	Patrol movement with easing types, ghost trail, gizmo path
BoostOrb.cs	Done	Collected by one character, boosts the other - speed/jump/both types
ExitDoor.cs	Done	Scale punch + settle + continuous pulse on activation
LevelManager.cs	Done	Tracks both exits, freezes characters on complete, timer integration
LevelTimer.cs	Done	Dual timers, active/inactive speed, frost icons, vertex wave, strobe
UIManager.cs	Done	Death flash + camera shake, label punch, triple-flicker dividing line
LevelCompleteUI.cs	Done	Bounce slide-in, elastic header pop, button fade
MainMenu.cs	Done	Title float, subtitle fade, button slide-up, fade in/out

Level Design Summary

Level	Core Challenge	Mechanic Introduced
1	Flat - learn Tab switching	Tab switch only
2	ARC activates switch, platform appears for ECHO	First WorldSwitch

Level	Core Challenge	Mechanic Introduced
3	Gaps + switch dependency	Gaps + switch
4	ECHO helps ARC for the first time	Reverse dependency
5	One-way gates	Platform Effector 2D
6	Two switches - order matters	Multi-switch
7	All elements + timer pressure	Full puzzle

Reflection

Where did AI excel, and where did it mislead or limit you?

Cursor was genuinely fast at the scaffolding work. Scripts that would have taken me days to write from scratch - PlayerController, CameraFollow, the singleton setup across LevelManager and CharacterSwitcher - were working within a few hours on Day 1. That time saving was a big deal in a three-day jam. Claude helped with documentation layout, giving me a structure I could fill in rather than building everything from nothing.

Where it fell short was harder to spot. The biggest problem was the BoostOrb coroutine bug. Cursor wrote code that looked completely correct - the logic made sense, it read cleanly - but the boost duration timer was attached to the BoostOrb MonoBehaviour. When the orb got destroyed on collection, Unity cancelled the coroutine before it finished, which meant the stat boost never got reversed and lasted forever. I only found it through actual play testing. The code passed a visual review because nothing about it looked wrong. That was a useful lesson: AI code that reads fine can still have lifecycle problems that only show up when things start destroying and creating objects.

Leonardo AI was a different kind of problem. The first batch of tiles came out too painterly and at irregular sizes - 297x249, 328x328 - not the clean 512x512 I needed. I had to run everything through birme.net just to get usable proportions. AI did not flag any of this. It just kept generating until the prompts were specific enough to work.

How did AI alter your creative or technical process?

The biggest shift was in where my time actually went. In a normal workflow, most of Day 1 would have gone to writing syntax, wiring up input systems, chasing NullReferenceExceptions. With Cursor handling that scaffolding, I spent that time reading and understanding code instead of producing it, and making actual design decisions - which mechanic to use, how to structure the levels, what the timer system should feel like in play.

The concept phase moved faster too. I got through five ideas in the same time it normally takes me to develop one properly. But that speed has a catch I noticed quickly: AI-generated ideas can sound solid without actually being solid. The switch from simultaneous mirrored control to the relay mechanic came from me playing the first version and finding it frustrating, not from AI feedback. AI told me the mirror mechanic was technically interesting. Playing it for five minutes told me it was annoying. That judgment call had to be mine.

The debugging loop got faster as well. When something broke I could paste the error into Cursor and get a targeted fix. But I made myself trace through every solution before accepting it. If I could not explain what the fix did, I did not use it. That slowed things down slightly but it meant I actually understood the codebase at the end of three days.

What would you change about your collaboration with AI next time?

The .cursorrules file helped a lot, but I would spend more time on it before writing any code. I ended up adding constraints mid-jam - using rb.linearVelocity instead of rb.velocity, CompareTag for character checks, the singleton pattern - because Cursor's defaults did not match Unity 2022.3 LTS. Getting those rules right upfront would have saved me a few rounds of corrections on the early scripts.

I would also ask better questions from the start. Something like "what happens to this coroutine if the object running it gets destroyed?" would have caught the BoostOrb bug before it ever reached testing. That kind of edge case is completely predictable - I just did not think to ask about it. Going forward I want to treat any AI-generated code involving coroutines, object destruction, or singleton lifecycles as something that needs an explicit edge case question, not just a visual check.

Ethical considerations: Did your use of AI respect originality, transparency, and fair use?

Yes. The core design of FLIPSIDE - the relay mechanic, the dual-timer system, the cross-character boost idea, the split-screen format, all seven levels - came from me. I used AI to build things faster and to stress-test ideas, not to generate the design itself. When Claude produced documentation structure I went through it section by section and made sure it actually reflected what happened, not what sounded good on paper. When Cursor produced scripts I tested them, traced through the logic, and in several cases changed them significantly before they went into the project.

Every AI contribution is logged in refinements-changes.md with the actual prompt I used and an honest note on what the output got right or wrong. Nothing in this project is presented as original human work when it was actually produced by AI without review and modification.

Did you credit AI-generated assets appropriately?

Yes. The credits table in readme.md lists every tool with a description of exactly what it contributed. The requirements.txt file lists every script with its AI role and what I changed

manually. Every art asset has its source tool logged, along with the model settings and a reference to the full prompts in refinements-changes.md. Every audio asset has its source tool. Nothing appears in the project without an attribution entry somewhere in the documentation.

Were there moments where relying on AI felt ethically questionable?

One moment in particular. When Claude generated the initial documentation structure - the plan.md layout, the log entry format, the readme sections - the documents looked complete enough to just submit as they were. That felt off. The documentation was supposed to be a record of my process, but it was written before most of the process had happened.

I dealt with it by treating everything as a template and going back to fill in the actual content after the work was done. The BoostOrb bug entry, for example, only went into refinements-changes.md after I found and fixed it - not before. The risk with AI-assisted documentation is that it produces something that looks like rigorous record-keeping without any real record underneath it. Keeping that distinction clear was probably the most important ethical decision I made across the whole jam.

How can you ensure your future work with AI remains responsible and authentic?

The logging approach I used in refinements-changes.md is the main thing I want to carry forward - prompt recorded, output assessed honestly, manual changes noted. It creates an actual paper trail of what decisions I made and why, rather than just a polished final product with no visible process behind it.

Beyond that, I want to hold the same line on understanding before accepting. Every AI script I used got traced through manually before it went into the build. It takes longer than just running it and seeing what happens, but code I can not explain is code I do not really own. That feels like the right standard to hold regardless of deadline pressure.

I will also keep being transparent about AI use in any professional or academic context. Not just because it is required, but because the work of designing how a human and an AI collaborate is itself something worth showing. The clearest evidence that I actually made this game is being able to walk through every decision - what I chose, what I rejected, and why.

References

Anthropic (2024) *Claude* [Large language model]. Available at: https://claude.ai (Accessed: March 2026).

Cursor (2024) *Cursor AI-assisted IDE* [Software]. Available at: https://cursor.sh (Accessed: March 2026).

Leonardo AI (2024) *Leonardo.Ai* [Image generation platform]. Available at: https://leonardo.ai (Accessed: March 2026).

Soundraw (2024) *Soundraw AI music generator* [Software]. Available at: https://soundraw.io (Accessed: March 2026).

ElevenLabs (2024) *ElevenLabs AI audio generation* [Software]. Available at: https://elevenlabs.io (Accessed: March 2026).

Unity Technologies (2022) *Unity 2022.3 LTS* [Game engine]. Available at: https://unity.com (Accessed: March 2026).